Software Engineering Semester Project

Software Analysis and Design (SSAD) | Fall 1405

Develop the minimum viable product (MVP) for the product your team has already selected and branded. Analyze and design it, build a working web application, test its main behavior, deploy it, and explain how it works.

An **MVP** is the smallest working version that solves a real user problem. This assignment develops your ability to turn user needs into software, justify design choices, direct AI tools, and verify a deployed product.

1 The project and your team

Your final product must provide:

- One complete business journey, such as booking, requesting, registering, submitting, or managing a record. Login alone or a decorative dashboard is not enough.
- A usable frontend connected to an API/backend and a database. Data must remain available after refreshing the page or returning later.
- Working login and logout, with backend rules that control access to protected actions and records.
- Meaningful tests and a working online version using HTTPS at a domain or an approved subdomain.

AI use is allowed throughout the project. You must understand, test, and explain the software you submit. AI use itself does not reduce your grade. Feature count, code volume, and technical complexity do not earn marks by themselves.

Team responsibilities

In a two-person team, one member usually leads frontend work and the other leads backend and data work. Both members must help define requirements, integrate the parts, test the product, and understand the whole system. In a three-person team, the third member must own real engineering work, rather than only slides or documents. Solo projects may have fewer features but must demonstrate the same core concepts.

The example used in this assignment

CampusMeet is a fictional example, not your assigned product. A university student views a professor’s available times, books an appointment, and later views or cancels their own booking. Its MVP excludes payments, complex calendars, notifications, and native mobile apps. These exclusions apply only to the example.

There are three formal presentations: analysis and design (20 points), development progress (25 points), and the final MVP with individual defense (55 points). Section 8 gives the dates, submission cutoffs, and assessment criteria.

2 Scope and user stories

Before designing screens or writing product code, describe who uses your product, the problem they face, what the MVP solves, and which features you will leave out. Keep the product idea and brand your team has already chosen.

For CampusMeet, students and professors need a clear way to arrange meetings. The MVP lets students book available slots instead of coordinating through messages.

Write stories and acceptance criteria

A **user story** describes a user's goal. Write the stories you can currently identify for the planned product and MVP using this format: "As a [user], I want [action], so that [benefit]." You are not expected to predict every future requirement.

CampusMeet US-03: As a student, I want to book an available time slot, so that I can meet my professor without sending messages.

Acceptance criteria are observable conditions that show a story works. For US-03:

- The student can see available slots and book one.
- The booking is saved and visible when the student returns.
- The same slot cannot be booked twice.
- A student cannot change another student's booking.

Give each relevant story clear criteria, including important failure or ownership cases. For example, a request for a slot that has just been booked must fail without creating a second booking.

Prioritize the scope

Mark essential stories **Must Have** and optional extras **Could Have**. You may use **Should Have** for useful but nonessential features. Complete the approved Must-Have scope before adding extras. Bring the product overview, prioritized stories, and acceptance criteria to Presentation 1.

3 Design the system

A web application connects what the user sees to rules and stored data. These terms describe the main parts:

- **Frontend:** the screens, forms, and buttons the user sees and uses.
- **UI component:** a reusable part of the user interface, such as an input field or a booking button.
- **Backend:** code that checks rules and permissions and reads or saves data.
- **API (application programming interface):** the agreed way the frontend asks the backend to do something and receives an answer.
- **Database:** storage that keeps information available after the current screen or visit.

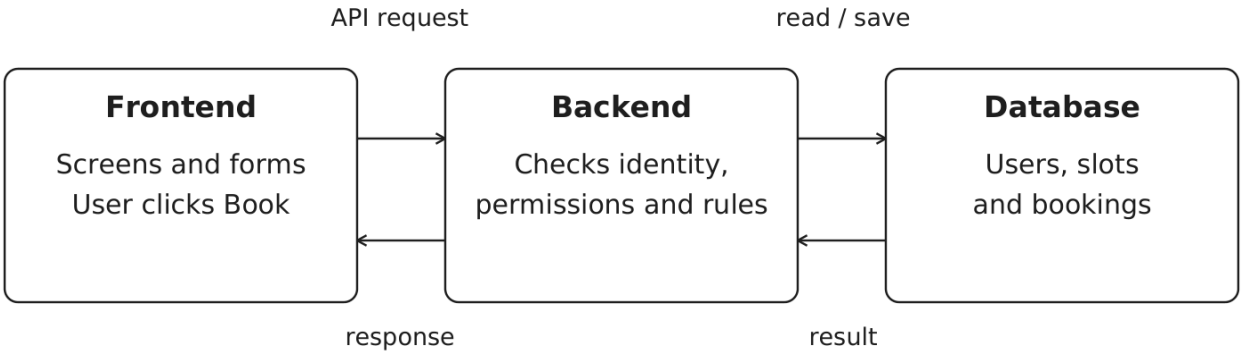


Figure 1 A Book click sends POST /bookings with a slot ID. The backend checks access and availability, saves the booking, and returns booking details or an error for the frontend to show.

Every student must be able to explain the UI action, API request, backend function, data read or saved, and response shown to the user for a real feature in their own project.

Choose and justify the architecture

Architecture is how you organize the software's parts and responsibilities. A framework is a tool used to build those parts. Choose and explain both your frontend and backend architecture; listing framework names is not enough.

A simple starting point is:

- **Frontend:** pages built from reusable components, with an organized place for API calls. Group related features and explain how forms, screen state, and server responses are handled.
- **Backend:** API routes call functions for business rules, which use database access code. Explain where authentication, authorization, input validation, data storage, and runtime configuration belong. Validation checks whether input is acceptable; runtime configuration supplies settings such as a database address.

These layers can remain in one application. Choose a structure your team can build and explain within the semester. CampusMeet might use a relational database because users, time slots, and bookings are linked. Explain the benefits and trade-offs of your own choice.

During design, check whether your frontend, backend, and database choices fit the product, expected usage, hosting platform, learning time, and budget. State the expected hosting cost and relevant limits. No programming language, framework, database product, authentication product, AI model, repository host, or cloud provider is prescribed.

Optional infrastructure must solve a clear problem. A **cache** keeps frequently needed results for faster reuse; a **message queue** holds work for later processing. Neither is required. Microservices, Kubernetes, and advanced infrastructure are also optional.

4 Prepare the design review

Complete the initial analysis and design before product development. Presentation 1 reviews these plans; no running product features are required. Small exploratory code may help you check feasibility, but it does not replace the design work.

Prepare four design views

UML (Unified Modeling Language) is a common notation for showing software behavior. Only two UML types are required here. Together with the system diagram and data model, they form the four required design views:

Design view	What to show
UML Use Case	The main actors (users or external systems) and the actions they can perform in the MVP.
UML Activity	The steps and decisions in one important workflow, including a failure or alternative path.
System architecture	The frontend, backend/API, database, relevant services, and how they communicate.
ERD or data model	The main stored entities or tables, essential relationships, and relevant ownership or constraints.

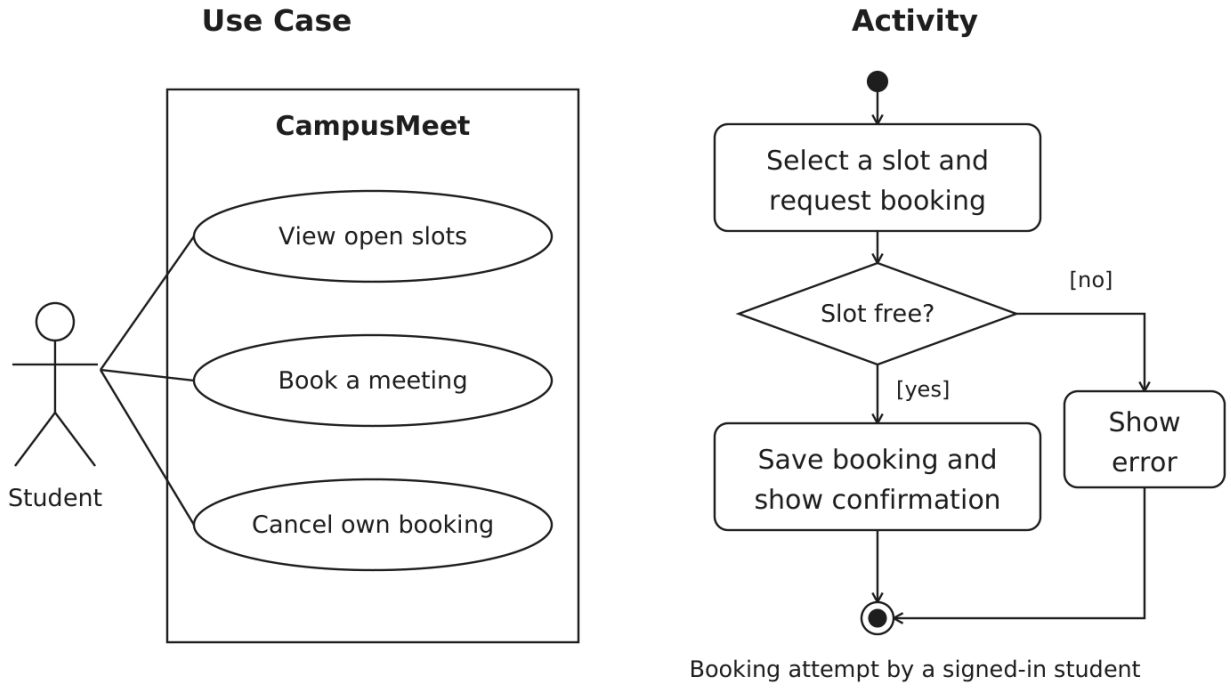


Figure 2 Two UML views of CampusMeet. Ovals show user goals; the activity view shows a booking attempt and its two outcomes.

Usually one clear view of each type is enough. Keep each view specific to your product. Formal C4 notation, class diagrams, sequence diagrams, and state-machine diagrams are not mandatory.

Model persistent data

An **ERD (entity–relationship diagram)** shows the data you store and how records connect. Design the main data model before coding; adapt the view if your storage model is not relational.

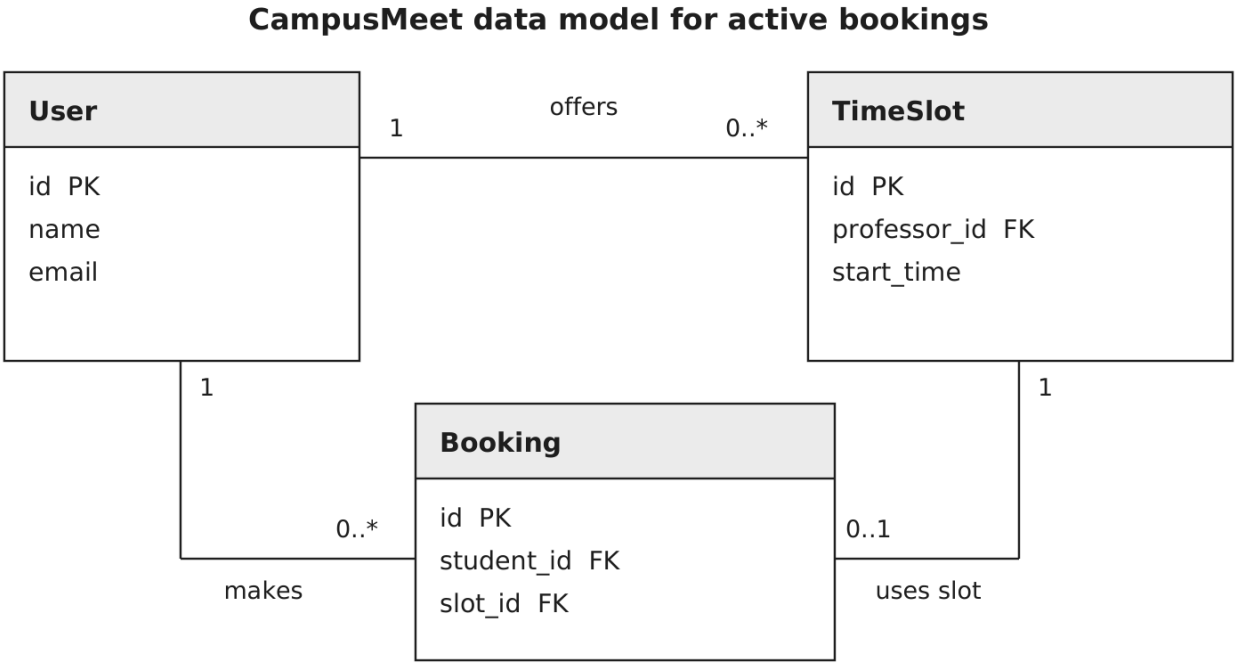


Figure 3 PK means primary key, a record's unique ID; FK means foreign key, a link to another record. Relationship labels: 1 means one, 0..* means zero or many, and 0..1 means at most one.

In this example, a booking belongs to one student and one slot. The design must prevent two active bookings for the same slot and enforce ownership. Your data model needs the relationships and constraints that support your own MVP.

Prototype the main journey

A **prototype** is a model of the interface used to try the workflow before implementation. Use Figma, Subframe, or an equivalent tool to make the main journey clickable. Include the main screens and relevant login, success, and error states. It does not need pixel-perfect visual design.

For CampusMeet, the prototype follows login, viewing open times, choosing a slot, and seeing either a booking confirmation or a useful error.

Agree on the API

An **API contract** records the request and response the frontend and backend agree to use. For every important operation, list the method and path, input, output, and possible errors. A short table is enough; OpenAPI is optional.

In this example, GET reads data and POST sends a request to create a booking. These paths illustrate CampusMeet; your product needs its own operations.

Example request	Input	Successful response	Example errors
GET /slots	None in this simple example	Available meeting slots	Data temporarily unavailable.
POST /bookings	Selected slot ID	Saved booking details	No login; access denied; slot no longer free.
GET /my-bookings	Identity from the signed-in session	The current user's bookings	No login.

Agree on these details before the frontend and backend are built separately. Bring the prototype, data model, API list, technology rationale, and hosting plan to Presentation 1 along with the scope and other design views.

5 Build and test with AI

After the design review, set up the project in a shared **Git repository**, which records changes to code and project files. GitHub and GitLab are examples of hosting services, not required choices. Keep setup and test instructions in a **README**, the repository's short project guide.

Build one small complete feature first. For CampusMeet, this could be showing available slots from the database in the frontend. Connect UI, API, backend, and stored data early, then add features in small increments. Frontend and backend teammates must keep integrating their work.

Use AI for specific changes

You may use AI agents for design, coding, tests, and debugging. Give each request a clear task and expected behavior. For example, "Build my whole booking app" is too broad. A more useful request is:

"Implement GET /slots from our API list. Return only future available slots from the database. Add a test and explain the files you changed."

After each AI-generated change, read the important code, run the application, run the tests including a failure case, and fix mistakes before continuing. You remain responsible for the result and must explain the relevant code and decisions.

Implement login and permissions

Authentication establishes who is signed in. **Authorization** controls which actions or records that user may access. Both must be implemented and tested by Presentation 2.

Real login and logout must work. You may use a mature authentication library, service, or suitable open-source solution; you are not expected to build a password and session system from scratch.

The trusted backend must enforce permissions. Hiding a button in the frontend is not enough because a request can be sent without that button. Demonstrate an allowed action and rejection of protected actions by the wrong user or an unauthenticated user.

For CampusMeet, Student A can access their own booking. Student B must be rejected when trying to change it, and a user who is not signed in must be rejected when requesting a private booking.

Presentation 2 is a **Development Progress Review** of working software and progress. Login and permissions are the minimum required functionality; the review also requires at least one real product feature and working UI/backend/database integration. It is not a formal security audit.

Test throughout development

A **test** checks expected behavior against an actual result. Run tests as you build, rather than waiting until all features are finished. AI may help write tests, but you must execute and understand them.

Test type	CampusMeet example
Function or unit	Check a small rule, such as rejecting an invalid appointment time.
API or integration	Check that a booking request saves data and rejects forbidden requests.
User journey	Follow the full workflow: log in, book a slot, and see the saved result.

At minimum, demonstrate automated checks of important application/API behavior and access permissions, including meaningful failures. Also test the complete primary user journey, manually or with automation. No coverage percentage, testing framework, load-test target, or formal security-testing regimen is required.

Know when a feature is finished

Use this definition of done for each feature:

- The expected behavior and an important error path work.
- The frontend, backend, and database work together with reliable stored data.
- Important tests pass and the team understands the changes.

- The shared version runs and the README is current.

6 Deploy the MVP

Deployment puts your software on hosting that runs it online. A **domain or subdomain** is its address. **HTTPS** protects the connection between the browser and the application.

Publish the complete MVP over HTTPS at a domain or a subdomain approved by course staff. You do not need to buy a separate domain. Check that the hosted frontend, backend, and persistent data support the full main journey.

Explain the hosting choice, actual or realistically estimated cost, and limits against expected usage. Include limits of any free plan; free hosting is optional and no provider is required. If you choose a server, consider whether you can configure, maintain, back up, and pay for it.

Add simple deployment steps to the README and update the diagrams, ERD, and API list to match the delivered product. Aim for a usable interface, useful errors, and reliable persistence. Advanced deployment automation and operational tooling are optional; no numerical performance or uptime target is prescribed.

7 Semester work plan

Use this recommended schedule to organize the work. The three presentations and their submission cutoffs in Section 8 are the formal milestones. Aim to complete features by 17 Dey and freeze product code by 30 Dey so there is time to verify the release.

All dates are in the Jalali year **1405**. All submission times are **Iran time (UTC+03:30)**. Presentation days are not extra development days.

Dates in 1405	Planned work
9–15 Aban	Define the MVP and prioritize user stories.
16–20 Aban	Prepare Use Case, Activity, system diagram, and initial ERD.
21–24 Aban	Prepare the prototype, API list, technology choices, and hosting plan.
25–26 Aban	Presentation 1 Analysis and Design Review
27 Aban–7 Azar	Set up code, database, API, and the first UI-to-API integration.
8–14 Azar	Implement authentication, authorization, and the core workflow.
15–22 Azar	Test the integration and rehearse the demo.
23–24 Azar	Presentation 2 Development Progress Review
25 Azar–17 Dey	Complete approved Must-Have features; target completion by 17 Dey.
18–24 Dey	Deploy, fix bugs, and test the main scenarios.
25–30 Dey	Finish fixes and documentation; code freeze by 30 Dey.

Dates in 1405	Planned work
1–3 Bahman	Verify the final version; submit by 3 Bahman at 18:00.
5–6 Bahman	Presentation 3 Final MVP and Engineering Defense

8 Presentations and submission

The three formal presentations total **100 points**, divided **20 / 25 / 55**. Show real behavior and explain the design and code. Each member must understand the whole system, even when primary responsibilities differ.

Presentation 1 Analysis and Design Review

25–26 Aban 1405 (16–17 November 2026) | **20 points**

Submit by **24 Aban at 23:59**. Format: **8-minute presentation + 4-minute questions**.

Show the problem, MVP boundaries, prioritized stories and acceptance criteria; the Use Case and Activity diagrams, system diagram, and ERD; a clickable prototype and API list; and reasons for the technology, architecture, and hosting choices, including cost and feasibility. Explain how the parts fit together. No working product feature is required at this stage.

Assessment criterion	Points
Product and user stories	8
Diagrams and system/database design	7
Prototype, API, and choices	5

Presentation 2 Development Progress Review

23–24 Azar 1405 (14–15 December 2026) | **25 points**

Submit by **22 Azar at 23:59**. Format: **7-minute demo + 5-minute questions**.

Live-demo the working frontend, API/backend, and connected database. Show tested login and logout, the signed-in identity, backend-enforced permissions, and at least one real product feature from UI to persistent data. Show tests of the demonstrated functionality and permissions, explain the progress, and identify the next tasks. This is a development review, not a formal security audit.

Assessment criterion	Points
Working product and integration	10
Authentication and authorization	8
Tests, progress explanation, and understanding	7

Presentation 3 Final MVP and Engineering Defense

5–6 Bahman 1405 (25–26 January 2027) | **55 points**

Submit by **3 Bahman at 18:00**. Format: **12-minute demo + 6-minute defense**.

Demonstrate the deployed MVP over HTTPS at its domain or approved subdomain. Show completed Must-Have stories, the full main journey, reliable login and permissions, persistent data, important errors, and practical tests. Provide current diagrams, ERD, API list, README and setup/deployment instructions. Explain hosting cost and known limitations.

Each member must answer engineering questions individually, including how a screen action reaches the API, backend rule, stored data, test, and deployed product, and why the team chose its architecture and technologies.

Assessment criterion	Points
Working MVP	20
Quality and tests	12
Deployment	8
Clear current documentation	8
Individual understanding	7

Submit one package per team at each review

Use the channel announced by course staff. Submit the following together, with evidence appropriate to the review:

- The repository link, with a README explaining how to run and test, the required diagrams, the API list, and the prototype link.
- The exact commit or tag being presented, brief test evidence, and a live URL when available. A commit or tag identifies a specific saved version in Git; Presentation 1 does not require working features or test results.
- A short note naming each member’s contribution and any unfinished work.

No additional essays or standalone reports are required when the information is clear in the repository. Keep documentation limited to what is needed to run, understand, test, and demonstrate the project. This assignment defines the course rules; project files show your actual work.